

# Doc-Helper 项目介绍（路演版）

一句话定位：一个本地运行、浏览器访问的“文档处理智能助手”——把“多表合并、批量替换、数据清洗”这类重复的 Office 苦力活，变成点几下（或说一句话）就能完成的小工具。

## 一、我们要解决的问题

- 很多人不会用 Office / WPS 的复杂功能：合并几十个 Excel、跨文档批量替换关键词、统一格式.....明明很简单，却要到处找教程、记宏、装插件。
- 重复劳动耗时耗力，且容易出错。
- 更重要：很多文档涉及敏感数据，用户不敢上传到云端工具处理。

一句话：让不懂 Office 的人，也能安全、快速地把文档活干完。

## 二、解决方案：两条使用路径，互为补充

路径 1：直接调用工具（对小白最友好）

浏览器里选工具 → 上传文件 → 点“开始运行” → 下载结果。全程不依赖大模型，开箱即用。

路径 2：智能体聊天，现场生成新工具

侧边栏用自然语言描述需求，智能体自动选择已有工具；没有合适工具时，现场生成新工具（草稿确认制——生成后需用户手动“启用”才生效，杜绝失控）。

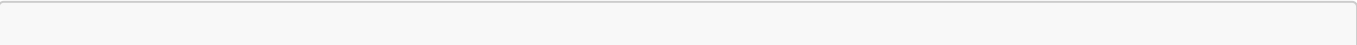
贯穿始终：本地运行，文档不出本机

所有处理都在本机完成，文件不上传任何服务器，天然满足数据隐私/合规诉求。

## 三、核心特性

| 特性        | 说明                                                     |
|-----------|--------------------------------------------------------|
| 🔌 工具即插即用  | 每个工具 = 一段 PowerShell 脚本 + 一份元数据（meta.json），可单独启用/禁用    |
| 🗨️ 智能体扩展  | 用自然语言让 AI 帮你“造新工具”，草稿确认制保证可控                           |
| 🔒 本地隐私    | 纯本地处理，无云上传，数据不出门                                       |
| 📦 零依赖文档引擎 | 内置 officecli 文档引擎，处理 .docx / .xlsx / .pptx，无需安装 Office |
| 🔗 前端插件化   | 界面同样采用插件架构（设置面板、聊天侧边栏、工具列表均为插件）                        |
| ✅ 质量保障    | 自动化测试覆盖注册/执行/集成等关键链路                                   |

## 四、系统架构（三层，清晰简单）



|     |                                     |
|-----|-------------------------------------|
| 顶层  | 智能体 + 界面                            |
|     | Node + LLM (OpenAI 兼容, 默认 DeepSeek) |
|     | 自然语言 → function calling → 调用工具      |
| 中间层 | 工具层 tools/                          |
|     | 一组 PowerShell 脚本 + meta.json 元数据    |
|     | 每个工具 = 一段脚本 + 一份声明                  |
| 底层  | officecli (内置 vendor/)              |
|     | 唯一文档处理引擎, 统一处理 Office 文档            |

**数据流：** 你在界面点按钮 / 聊天 → 后端调度 → 调用工具脚本 → 脚本驱动内置文档引擎干活 → 返回结果。

**技术栈：** Node.js 后端 + 原生 HTML/CSS/JS 前端（零构建链）+ PowerShell 工具脚本 + officecli 引擎 + LLM function calling。

## 五、技术亮点

- 1. **本地隐私优先：** 设计上就为“文档不出本机”而生，适合对数据安全敏感的用户。
- 2. **AI 可解释的工具规范：** 工具元数据采用 OpenAPI 风格，直接映射为大模型的 function calling schema——大模型“看得懂”每个工具，是智能体自动选工具/造工具的基础。
- 3. **草稿确认制：** AI 生成的工具默认“草稿”，必须用户手动启用，安全可控。
- 4. **工具执行安全红线：** 后端只按工具名查表执行已注册工具，绝不执行 LLM 返回的任意命令；路径防穿越、超时 120s、可取消。
- 5. **系统提示工程落地：** 为“生成工具”场景编写了完整指令模板并作为 system prompt 注入，让大模型能稳定产出合规工具。
- 6. **零构建的前端插件化：** 界面功能以插件形式组织，扩展界面能力不需要打包工具链。

## 六、当前进展（MVP 阶段 v0.1）

### 已完成 ☒

- 三层架构全部跑通，自动化测试 28 个用例通过
- 网页界面：工具列表、工具调用、拖拽上传、实时进度、结果下载
- 第一个真实工具：**xlsx 多表合并**（已验证可用）
- 智能体聊天：LLM 配置、function calling 循环、现场生成工具（草稿）+ 生成前端插件
- 工具规范 + 生成指令模板成型

### 进行中 / 待办

- 补齐工具集：Excel 拆分、数据清洗、格式调整、Word 批量替换、Word 多文档合并
- 界面打磨、细节完善

### 本期不做

- 多用户 / 账号 / 云端部署（当前纯本地）

- 移动端适配
- PPT 处理

---

## 七、快速演示脚本（30 秒 Demo）

1. 启动：`node server/index.js` → 浏览器打开 `http://localhost:3000`
2. **直接调用**：选"xlsx 多表合并" → 拖入 2 个 Excel → 点运行 → 看到进度 → 下载合并结果 ☒
3. **智能体造工具**（如已配置大模型）：侧边栏输入"做一个统计 Excel 每个工作表行数的工具" → 智能体生成草稿 → 点"启用" → 新工具出现在列表 ☒

---

## 八、下一步规划与愿景

- **短期**：完成 MVP 工具集（7 个：合并/拼接/拆分/格式/清洗/Docx 替换/Docx 合并），让日常文档活全覆盖。
- **中期**：打磨"智能体造工具"体验，成为低代码式文档处理平台：用户说需求，AI 造工具，社区可分享工具包。
- **愿景**：让每个人都能用自然语言"指挥"自己的文档，而不用学 Office 的复杂操作——一个**私有、安全、会自己长本事**的文档助手。

---

## 附：运行方式

```
node server/index.js          # 默认端口 3000
$env:PORT=8080; node server/index.js  # 换端口
```

需要电脑装有 Node.js（本项目使用 Node 26）；大模型为可选配置（工具直接调用不需要）。